How Poker Solvers Are Trained

A field guide to the ML behind equilibrium poker — written to start the first Drawmaha solver

research synthesis of 70 sources · August 28, 2026

Abstract. This survey explains how machine-learning poker solvers are actually built and trained: what "solving" a poker game means, why the standard applied-RL toolkit (reward functions, actor–critic, self-play) fails at it, and what the poker-AI community built instead — the counterfactual-regret family, its neural descendants, the search-plus-value-network lineage, and the modern RL methods that fix self-play's cycling. Every method is presented intuition-first, with its tradeoffs and its meaning for one concrete project: building the first solver for heads-up pot-limit Drawmaha, a split-pot draw/Omaha hybrid with 2,598,960 possible starting hands and no prior solver, bot, or academic mention as of August 2026. It closes with the full checklist of design decisions the project will face. Synthesized from an eight-lane literature sweep (about 70 verified sources); the cited originals remain ground truth.

TL;DR **— the ideas of the lecture**

- "Solving" means computing a **Nash equilibrium** — a strategy so balanced that even an opponent who knows it exactly cannot beat it. Its distance-from-perfect is a measurable number, **exploitability**.
- The entire field rests on one ten-line rule, **regret matching**, and one theorem: the *average* of self-play strategies converges to equilibrium while the current strategy cycles forever. Deploy the average, never the last iterate.
- **CFR** runs that rule at every decision point of the game tree; **MCCFR** samples the tree instead of walking it; **Deep CFR** replaces the regret tables with networks when the game no longer fits in memory. This is the project's spine.
- Naive PPO-style self-play has *no* equilibrium guarantee in poker — the dynamics provably orbit instead of converging. Every working RL fix restores convergence by averaging, populations, or regularization.
- Training loss means nothing here; the only honest progress meters are exploitability on small games and best-response probes plus variance-reduced head-to-head at full scale.
- Drawmaha adds three specific complications — a 2.6M-hand space, a draw round with a public draw count, and a split pot — and the literature has a precedent for each; the split pot is the cheapest of the three.

1 "Solving" means unexploitable, and unexploitable is a number

WHAT SOLVING MEANS

Machine learning usually optimizes against a fixed target: a loss goes down, accuracy goes up. Poker has no fixed target. The quality of any strategy depends entirely on what the opponent does, and the opponent adapts. So the field's definition of "solved" is borrowed from game theory: a strategy pair is a **Nash equilibrium** when neither player can improve by deviating unilaterally. An equilibrium strategy is one you could publish — an opponent who reads your exact strategy still cannot beat it in expectation.

Nash equilibrium — a pair of strategies where neither player can gain by changing only their own

Three classical facts make this the right target for *heads-up* poker, and they are worth internalizing before any algorithm:

1. **Two-player zero-sum is special.** Heads-up poker is **zero-sum** — every chip you win, the opponent loses. Von Neumann's minimax theorem says such games have a unique *value* (the expected win rate of perfect play), and all equilibria achieve it. Better: equilibria are *interchangeable* — any equilibrium strategy of yours is safe against any strategy of theirs, guaranteeing at least the game value. This is exactly what fails with three or more players, which is why every guarantee in this survey is heads-up-only (Section 14 returns to this).
2. **Equilibrium play must be random.** If you only bet when strong, your bets stop getting paid; if you never bluff, your checks get attacked. The equilibrium bluffs, calls, and value-bets with calibrated *frequencies* — it is a **mixed strategy**. This single fact — optimal play is deliberately stochastic — is what breaks most standard RL machinery later in this survey: greedy "take the best action" improvement has no fixed point to find.
3. **Distance from equilibrium is measurable.** Fix your strategy σ and let a perfect adversary — the **best response** — extract the maximum from it. What it wins above the game value is your **exploitability**. Exploitability is the loss function of this entire field: every algorithm in this survey is judged by how fast it drives that number toward zero, and Section 11 is about how to measure it at each scale.

zero-sum — whatever one player wins, the other loses; payoffs sum to a constant

mixed strategy — a probability distribution over actions, rather than one fixed action

best response — the strategy maximizing EV against one known, fixed opponent strategy

exploitability — how much a perfect counter-strategy beats your strategy for; zero exactly at equilibrium

The object being solved: a game tree with hidden cards

EXTENSIVE FORM

Poker is played on a tree: chance nodes deal cards, decision nodes belong to a player, leaves pay out the pot. The formal name is an *extensive-form game*. Hidden information enters through one definition. Suppose you hold $K\,K\,9\,5\,2$ after a bet; there are many tree nodes consistent with what you see — one per opponent hand — and you cannot tell them apart. That set of indistinguishable nodes is one **information set** ("infoset"), and a strategy must assign *one* action distribution per infoset, not per node. Everything downstream — what CFR tables index, what networks take as input, what the dashboard displays — is keyed by infosets.

information set — all game states a player cannot tell apart: own cards + public cards + the action history

> INTUITION — **hidden information means you play against a probability cloud**
>
> In chess you respond to *the* position. In poker you respond to a cloud: every opponent hand, weighted by how likely it is to have played this way — their *range*. An equilibrium strategy is one whose action frequencies hold up against the whole cloud at once. This is also why poker strategies are ranges, not hands: the dashboard question is never "what does he have?" but "what fraction of everything he could have does this?"

One technical condition matters enough to name: **perfect recall** — players remember everything they saw and did. Kuhn's theorem (1953) guarantees that under perfect recall, per-infoset randomization ("behavioral strategies") loses no generality, and CFR's convergence proof quietly depends on it. It becomes a live issue exactly once in this survey: "imperfect-recall abstraction" (Section 7) deliberately violates it to save memory, trading the guarantee away.

perfect recall — no player ever forgets their own past observations or actions

Heads-up Drawmaha fits this frame with no modification. The split pot makes leaf payoffs more intricate (halves, scoops), but payoffs still sum to the pot: the game is constant-sum, equivalently zero-sum, and the minimax/interchange-

Drawmaha's split pot changes payoffs at the leaves only — the game stays zero-sum, and every guarantee in this section applies unchanged.

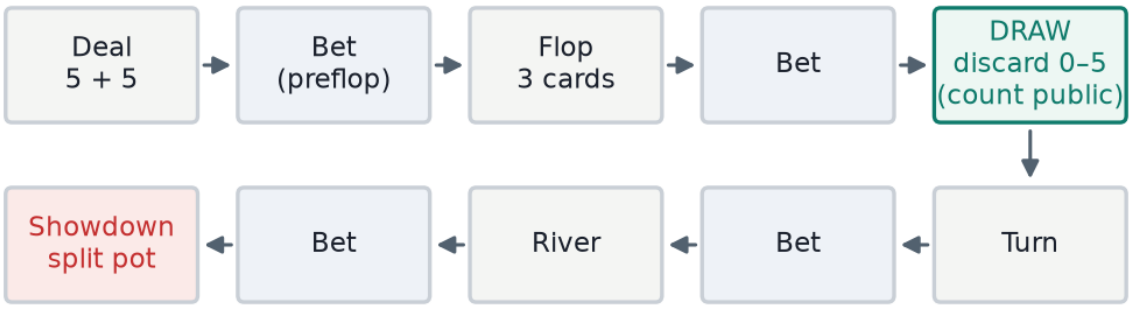


Figure 1: The street structure of the target game: four betting rounds, one draw round, split-pot showdown.

ability/exploitability story carries over exactly.

2 Drawmaha: the exact game, and the three numbers that shape everything

THE GAME SPEC

Solver guarantees are about one exact ruleset, so the target game must be pinned before any algorithm talk — Drawmaha has house variants (draw timing, betting rounds, declare rules), and several design choices later in this survey silently depend on which one is meant. This project's spec:

- **Heads-up**, blinds, **pot-limit**; version-1 action menu {fold, check/call, pot}, with the bet-size menu a config so multi-size solves are a re-solve, not a rewrite.
- Each player is dealt **five hole cards**. Betting; a three-card **flop**; betting; **one draw** (discard 0–5, replace); a **turn** and betting; a **river** and betting; showdown. Four betting rounds total, one draw round (Figure 1).
- **Split pot, cards speak**: half to the best five-card poker hand formed by the hole cards themselves (the *inner hand*), half to the best Omaha-style hand using exactly two hole cards plus three board cards (the *outer hand*). One player can *scoop* both halves.
- **Draw-count is public; identities are hidden.** Both players see how many cards each drew, never which. House rule adopted: **drawing exactly one** exposes the replacement face up, and the drawer may keep it or reject it for a face-down second card — an extra public signal and an extra decision node unique to this project.

Why this game is hard: three separate blow-ups

THREE BLOW-UPS

Comparing against the games the literature has already conquered isolates exactly what is new. Drawmaha combines three complications that existing work has faced only one at a time:

	Hold'em	Pot-limit Omaha	**Drawmaha**
Starting hands	1,326	270,725	**2,598,960**
after suit isomorphism	169	~16,500	**134,459**
Draw rounds	0	0	**1** (32 discard subsets)
Pot	single	single	**split** (inner + outer)
Betting rounds	4	4	4

Blow-up 1: the hand space. $\binom{52}{5} = 2{,}598{,}960$ private hands per player — roughly 2,000× hold'em. Dealing both players runs to $\binom{52}{5}\binom{47}{5} \approx 4.0 \times 10^{12}$ deals before the board or draws multiply it. Every table, buffer, range vector, and dashboard grid in this survey has to survive this number; it is why the project's endgame is a network rather than a table (Section 8), and why a raw DeepStack-style range input does not port (Section 9). The one free win is **suit isomorphism**: relabeling suits collapses the 2.6M hands to **134,459** canonical classes — a 19.3× reduction that costs nothing (Section 7).

suit isomorphism — hands identical up to relabeling suits play identically; merging them is provably lossless

Blow-up 2: the draw round. The draw inserts a player-*chosen* chance event mid-hand: choosing which of $2^5 = 32$ subsets to keep is an action; the replacement cards are chance. Its strategic signature is that the draw count is public information — standing pat announces strength (or a bluff, the classic "snow"), and drawing three announces weakness — so both players' full draw-count histories belong in the infoset. The closest solved precedents (2-7 triple draw, badugi) are commercial, not academic; their one loud lesson is that discard *identity* (which cards you threw) drives blocker effects worth real EV (Section 12).

Blow-up 3: the split pot. Cheapest of the three, and the literature says exactly why: a split pot changes only the *terminal payoff function* — who gets how much at showdown — and never the information structure of the tree. Any CFR variant runs unmodified once the leaves pay out correctly; the engineering moves into a fast dual evaluator (inner-hand rank and outer-hand rank per showdown) and into strategy itself becoming two-dimensional: a hand now has inner equity, outer equity, and a scoop probability, which haunts abstraction (Section 7) and evaluation units (Section 11).

One more fact frames the whole project: the literature sweep behind this survey searched academic and commercial channels for any Drawmaha solver, bot, or analysis and found **nothing** — the game is documented (rules sites, online mixed games) but unstudied. The "first Drawmaha solver" claim stands as of August 2026, with a consequence for evaluation: there is no external baseline to beat, so the project must manufacture its own opponents (Section 11).

Searched: Semantic Scholar, arXiv, GitHub, commercial solver market, poker forums. Nothing for "Drawmaha" or "Sviten Special."

3 Regret matching: the ten-line rule the whole field runs on

Strip poker away entirely and start with rock–paper–scissors. The engine underneath every solver in this survey is a 2000 learning rule by Hart and Mas-Colell called **regret matching** [1]. After each round, ask for every action: *how much better would I have done had I always played that instead?* Add that gap — the **regret** — to a running ledger per action. Next round, play actions in proportion to their positive ledger entries.

```
ledger = {a: 0 for a in ACTIONS}            # cumulative regret per action

def strategy():                              # regret matching
    pos = {a: max(r, 0) for a, r in ledger.items()}
    Z = sum(pos.values())
    return {a: p/Z for a, p in pos.items()} if Z > 0 else uniform()

def update(my_action, their_action):
    for a in ACTIONS:                        # the what-if ledger update
        ledger[a] += payoff(a, their_action) - payoff(my_action, their_action)
```

REGRET MATCHING

regret matching — play each action with probability proportional to its accumulated positive regret

regret — hindsight gap: (what action a would have paid) − (what I actually got)

Rail trace — round 1, I throw Rock, they throw Paper (payoff −1):
what-ifs: Paper → 0, Scissors → +1.
ledger: P += 1, S += 2, R += 0
next play: P $\frac{1}{3}$, S $\frac{2}{3}$, R 0.
Round 2, I throw Scissors, they hold Paper (+1): ledger → R −2, P 0, S +2 — all-in on Scissors, and the chase begins.

Notice what is *absent*: no learning rate, no optimizer, no gradient — accumulate and normalize. Hart and Mas-Colell proved this rule is a **no-regret learner**: its average regret vanishes over time no matter what the opponent does. That

no-regret learner — average regret per round provably shrinks toward zero as play continues

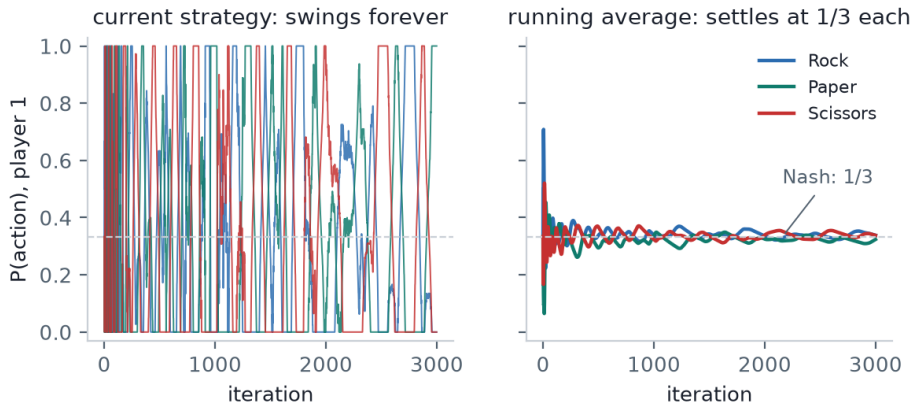


Figure 2: Both players running regret matching in rock–paper–scissors (simulated for this paper): current strategy vs. its running average.

property, not any poker insight, is the load-bearing theorem of computer poker.

Because when *both* players run regret-matching against each other, two things happen at once, and Figure 2 shows them with a real simulation. The *current* strategies chase each other forever — I over-throw Scissors, you learn Rock, I learn Paper, and the ledgers keep circling. But a folk theorem of game theory converts vanishing regret into equilibrium: in a two-player zero-sum game, if both players' average regret is ε, the pair of *time-averaged* strategies is a 2ε-Nash equilibrium. The orbit never stops; the *center of the orbit* is the answer. In the figure the running average settles onto exactly $\frac{1}{3}, \frac{1}{3}, \frac{1}{3}$ — the unexploitable RPS strategy — while the current strategy never settles at all.

The single most common conceptual bug in this field is deploying or measuring the current strategy when every guarantee is about the average. It will reappear in CFR (query the average policy), in Deep CFR (the entire reason an average-policy network exists), and in the RL section (methods that fix cycling by construction so the last iterate *does* converge). Keep the distinction sharp now and the rest of the survey reads easily.

4 CFR: a little regret-matcher at every decision point

COUNTERFACTUAL REGRET

Regret matching handles one simultaneous decision. Poker is a *tree* of sequential decisions under hidden information, and the naive fix — treat every complete game plan as one giant "action" and regret-match over plans — explodes combinatorially. The 2007 breakthrough of Zinkevich, Johanson, Bowling, and Piccione [2] is **counterfactual regret minimization**: put an *independent* little regret ledger at every information set, define each one's regret the right way, and prove the sum of the local regrets upper-bounds the global regret of the whole strategy. Minimize locally, win globally — then the folk theorem from Section 3 converts the vanishing regret into a converging *average* strategy.

CFR — run an independent regret-matcher at every infoset; their local regrets provably bound global regret

The one subtlety is what "the right way" means, and it is where the name comes from.

> INTUITION — **counterfactual value: score the spot as if you had steered to it**
>
> An infoset deep in the tree is only reached some of the time, and partly because of *your own* earlier choices. If local regrets were weighted by how often *you* actually go there, a spot you currently avoid would accumulate no regret

and never be reconsidered — learning would freeze. So CFR scores each infoset *counterfactually*: weight its value by the probability that *the opponent and the deck* let you arrive (their reach), while pretending you steered there with probability one. Your own choices never gate their own reconsideration; the opponent's play and the cards set the stakes. That reach-weighting detail is the entire trick — and getting it backwards is the classic silent implementation bug (Section 13).

In symbols, for infoset I with strategy σ_t at iteration t, action a's instantaneous regret is

$$r_t(I,a) \;=\; \pi_{-i}^{\sigma_t}(I)\,\Big[\,v(I,a) - \textstyle\sum_{a'} \sigma_t(I,a')\,v(I,a')\,\Big],$$

where $\pi_{-i}^{\sigma_t}(I)$ is the opponent-and-chance reach probability and $v(I,a)$ the expected payoff after taking a at I. Each ledger accumulates $R_T(I,a) = \sum_t r_t(I,a)$; regret matching over R_T^+ gives the next strategy; and the guarantee is exploitability of the *average* strategy shrinking as $O(1/\sqrt{T})$.

```
def cfr(state, reach_me, reach_opp):          # one traversal, my update
    if state.terminal(): return state.payoff()
    I = infoset(state)                        # my cards + public history
    sigma = regret_match(ledger[I])
    v = {a: cfr(state.play(a), ...) for a in I.actions}   # recurse
    ev = sum(sigma[a] * v[a] for a in I.actions)
    if I.player == me:
        for a in I.actions:                   # counterfactual weighting
            ledger[I][a]     += reach_opp * (v[a] - ev)
        avg_ledger[I][sigma] += reach_me      # average strategy, own reach
    return ev
```

KUHN POKER

Kuhn poker — 3-card deck (J,Q,K), one card each, one bet of 1; the smallest real poker game

The standard first proving ground is **Kuhn poker**: twelve infosets, yet genuine hidden information, value betting, and an equilibrium that *must* bluff (the jack bluffs one-third as often as the king value-bets). Its equilibrium is known in closed form, so an implementation can be checked to the digit. Figure 3 shows CFR solving it live for this paper, and the plot restates every claim of the last two sections with real data: the average strategy's exploitability marches down the $1/\sqrt{T}$ guide toward zero; the *current* strategy of vanilla CFR stays stuck around 0.2 chips/hand forever, cycling exactly as the theory predicts.

The simulation behind Fig. 3 reproduces Kuhn's game value $-1/18 \approx -0.0556$ for the first player to 4 decimal places — the kind of closed-form check Section 13 argues every implementation needs.

The figure also previews the next section: the teal curves are **CFR+**, a three-tweak variant whose *current* strategy ends up less exploitable than either average — the empirical surprise that made million-fold-larger games solvable.

Two closing notes calibrate expectations. First, cost: vanilla CFR walks the *entire* tree every iteration — fine for Kuhn's 58 nodes, unthinkable for Drawmaha's 10^{12} deals; Sections 5–6 are the escape routes. Second, memory: CFR stores two numbers per infoset-action (regret sum, strategy sum). That linear-in-infosets footprint is precisely what made 10^{12}-state abstractions of limit hold'em solvable in 2007 where LP solvers managed 10^8 — and precisely what runs out at Drawmaha scale, forcing Section 8.

5 The tabular speedups: CFR+, discounting, pruning, vectorization

CFR+

Everything after 2007 is an efficiency layer on the same skeleton, and the layers compound. They matter to this project twice: the toy-game and mini-drawmaha phases will literally run them, and Deep CFR silently inherits their weighting

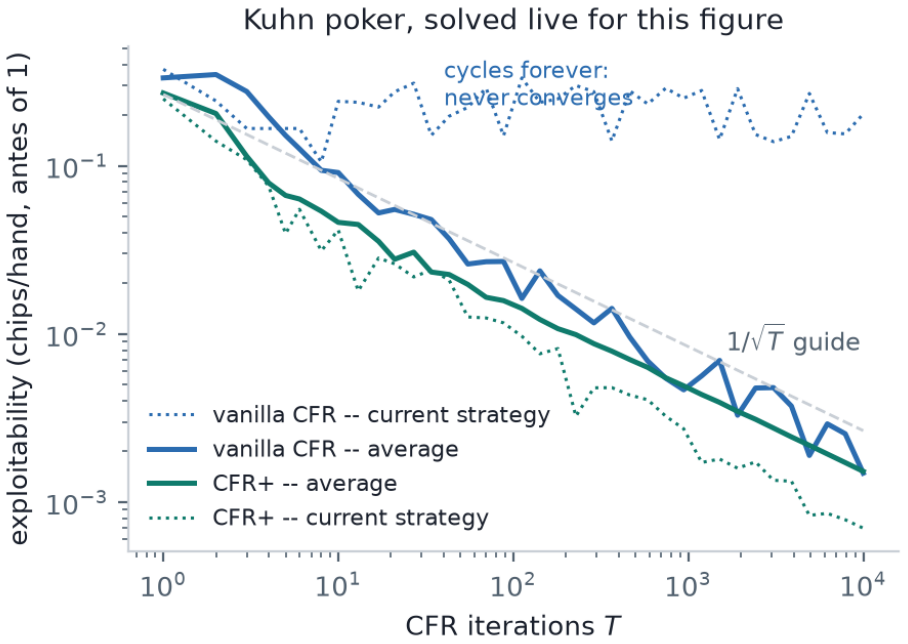


Figure 3: Real CFR runs on Kuhn poker (computed for this paper): average-strategy exploitability converges; current strategy either cycles (vanilla) or, under CFR+, converges on its own.

ideas.

CFR+ (Tammelin 2014) [5, 6] is three small changes applied *together*: (1) *regret-matching+*: clip each ledger entry at zero after every iteration, so an action buried under old negative regret can come back the moment it turns good, instead of slowly paying down debt; (2) *alternating updates*: update one player per traversal, so each responds to the other's freshest strategy; (3) *linear averaging*: weight iteration t's contribution to the average by t, washing out the garbage early iterations. The provable rate is unchanged ($O(1/\sqrt{T})$), but empirical convergence often looks like $1/T$ or better — visible in Figure 3, where CFR+'s *current* strategy beats every average curve. CFR+ is what solved full heads-up limit hold'em: **Cepheus** (2015) [7] drove a 10^{14}-infoset game below 1 milli-big-blind/hand of exploitability using 4,800 CPUs for 68 days — both the triumph of the tabular approach and the clearest statement of its ceiling. Cherry-picking one of the three tweaks (say, clipping alone with uniform averaging) can converge slower than vanilla CFR — they are specified as a package.

Discounted CFR (Brown & Sandholm 2019) [8] generalizes the idea: scale positive regrets by $t^\alpha/(t^\alpha+1)$, negative by $t^\beta/(t^\beta+1)$, average-strategy contributions by $(t/(t+1))^\gamma$. The recommended $(\alpha, \beta, \gamma) = (1.5,\ 0,\ 2)$ beat CFR+ in every game tested — the first algorithm to do so. Its sibling **Linear CFR** (everything weighted by t) matters for a different reason: it is the member that stays sound under *sampling*, and Deep CFR builds on exactly that weighting. The rule of thumb the literature converged on: full traversals → DCFR$(1.5, 0, 2)$; anything sampled → linear weighting.

Three more tools complete the tabular kit:

- **Regret-based pruning** (Brown & Sandholm) [9]: temporarily skip traversing subtrees hanging off actions with deeply negative regret — they contribute nothing to the current strategy — and revisit them occasionally in case they recover. Large constant-factor savings; used inside the Libratus/Pluribus blueprints.
- **Vector-form / public chance sampling** (Johanson et al. 2012) [10]: traverse the

public tree once while carrying a vector of values for *every* private hand simultaneously — turning thousands of scalar traversals into one matrix-shaped pass. This is the trick behind PioSolver-class speed and behind fast exact best-response computation (Section 11). For Drawmaha the private-hand vector is 134,459 entries (canonical classes) instead of hold'em's 1,326 — painful but not absurd on a GPU.

- **Sequence-form LP** (Koller, Megiddo & von Stengel 1994–96) [11]: small games can skip iteration entirely — an exact linear program solves them to machine precision (the pre-CFR state of the art, alive in the Gambit software). Its modern role is generating *ground truth*: the Leduc value used as a regression constant in Section 13 comes from exactly this.

Variant	The trick	Use it when
Vanilla CFR	the 2007 skeleton	learning; debugging
CFR+	clip + alternate + linear avg	full traversals, toy games
DCFR$(1.5, 0, 2)$	tuned discounting	full traversals, best-in-class
Linear CFR	weight everything by t	anything sampled (incl. Deep CFR)
Pruning	skip hopeless subtrees	big trees, blueprint solves
Vector-form	all hands in one public pass	exact solves, best response
Sequence-form LP	exact, no iteration	tiny games; ground truth

None of this changes the memory wall: every variant above still stores ledgers for every infoset. Two independent escapes exist — sample the tree so cheap iterations replace exhaustive ones (next section), and shrink or generalize the infoset space itself (Sections 7–8).

Project translation: Kuhn/Leduc run vanilla (to learn) then CFR+/DCFR (to go fast); mini-drawmaha runs sampled linear-weighted MCCFR; Deep CFR inherits the linear weights.

6 MCCFR: learn from dealt hands instead of enumerating the universe

MONTE CARLO CFR

Vanilla CFR's iteration cost is the whole tree, and Drawmaha's tree is dominated by chance: $\approx 4.0 \times 10^{12}$ ways to deal the two starting hands, before boards and replacement cards. **Monte Carlo CFR** (Lanctot et al. 2009) [4] replaces "enumerate every deal" with "play sessions": each iteration samples a slice of the tree and updates only the infosets it visits, using estimates constructed to equal the full-CFR update *in expectation*. Convergence survives (with high probability); iterations get cheaper by orders of magnitude; you take many more of them and win on wall-clock.

MCCFR — sample part of the tree each iteration; update visited infosets with estimates that are correct on average

The design space is *what to sample*, and Figure 4 draws the two poles:

- **External sampling**: deal one random hand, sample the opponent's actions from their current strategy, but at *your own* decision points try every action. One iteration costs about one deal plus playouts — and because everything "external" to you was sampled at its true probability, the estimates need **no importance-weight corrections at all**. Low variance, simple, the practitioner default.
- **Outcome sampling**: sample one complete playout including your own actions. Cheapest possible iteration — and the estimate must be divided by the probability of having sampled that path, an **importance weight** that explodes as strategies sharpen and probabilities shrink. High variance is the price of touching one path.

external sampling — sample chance + opponent choices; traverse ALL of the updating player's own actions

outcome sampling — sample a single playout, everything included; correct with importance weights

importance weight — 1/(probability of the sampled path): rescales a sampled outcome so its average is unbiased

> INTUITION **— importance weights are an honest bookkeeper with a loud voice**

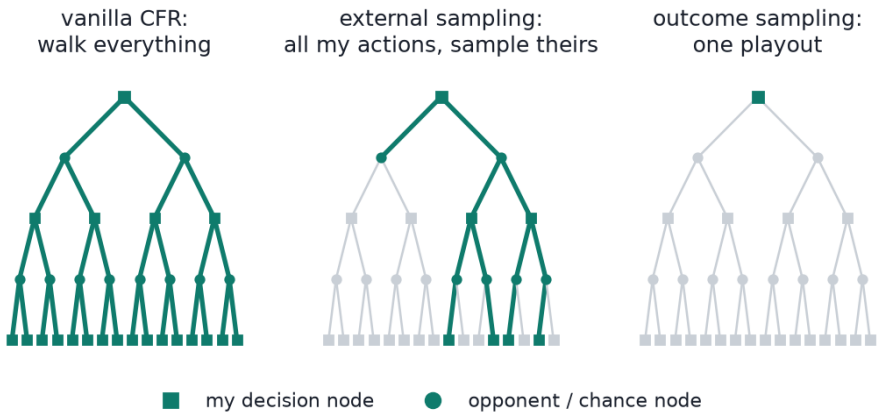


Figure 4: What one iteration touches under each traversal scheme (teal = visited).

If you only visit a spot 1% of the time, the rare visit must count 100× to keep the average honest — that division by the sampling probability is all an importance weight is. The bookkeeping is exactly right *on average* and horribly noisy per sample: one rare visit slams the ledger. External sampling's quiet superpower is never needing the correction for your own actions (you tried them all), which is why the variance stays tame — and why Deep CFR chooses it (Section 8).

```
def traverse(state, me):                    # external-sampling MCCFR
    if state.terminal(): return state.payoff(me)
    I = infoset(state); sigma = regret_match(ledger[I])
    if I.player == me:                      # my node: try EVERY action
        v = {a: traverse(state.play(a), me) for a in I.actions}
        ev = sum(sigma[a] * v[a] for a in I.actions)
        for a in I.actions:
            ledger[I][a] += v[a] - ev       # no importance weights needed
        return ev
    else:                                   # opponent/chance: SAMPLE one
        a = sample(sigma if I.player else state.chance_dist())
        avg_ledger[I][sigma] += 1           # opponent's avg-strategy bookkeeping
        return traverse(state.play(a), me)
```

For an RL-trained reader, one bridge makes the whole variance topic familiar: **VR-MCCFR** (Schmid et al. 2019) [12] subtracts a learned per-infoset *baseline* from sampled values and adds it back in expectation — literally the control-variate/advantage-baseline trick from policy-gradient RL, imported wholesale. Reported effect: variance down ~ 1000×, an order-of-magnitude speedup, and the first working marriage of sampling with CFR+-style updates. It is an optimization, not a correctness requirement — the note for the project is simply that plain external sampling already converges.

if mini-drawmaha MCCFR converges too slowly, noise is the first suspect and baselines are the literature's fix.

Two practical rules close the section. If you own the simulator — and this project does — external sampling is the default; outcome sampling exists for black-box environments and pays a variance tax that takes a whole extra learned network to tame (DREAM, Section 8). And under sampling, use *linear* weighting rather than CFR+'s clipping (Section 5): clipping interacts badly with noise, a lesson that carries straight into Deep CFR's buffers. Middle grounds exist ("robust sampling" traverses k of your actions instead of all — a compute/variance dial worth remembering if Drawmaha's 32-subset draw node makes full external traversal expensive), but external sampling is where this project starts.

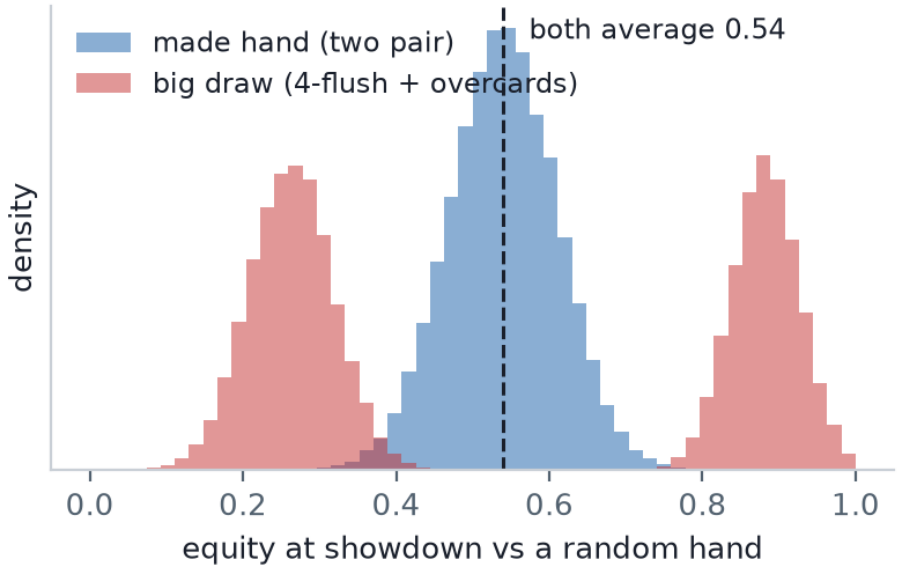


Figure 5: Two hands with equal average equity and opposite equity distributions (illustrative densities): the scalar cannot see the difference; the histogram can.

7 Abstraction: deciding which hands share a strategy

ABSTRACTION

abstraction — solve a smaller merged game whose solution transfers back to the real one

Sampling makes iterations affordable; the ledgers themselves still need one row per infoset. The classical answer is **abstraction**: merge strategically similar situations, solve the smaller game, lift the strategy back. A decade of poker research is essentially a ratchet of better answers to "which hands should share a row?" — and even though this project's endgame replaces tables with networks, the ratchet survives as the checklist for what the network's *inputs* must express, and as the recipe for the dashboard's bucketed range view.

Layer 0 — lossless. Some merges cost literally nothing: hands identical up to relabeling suits play identically, and Gilpin & Sandholm [13] proved equilibria of the merged game lift back *exactly*. For Drawmaha the sweep behind this survey brute-force-verified the payoff: the 2,598,960 five-card hands collapse to exactly **134,459** canonical classes (19.3×). Suit-isomorphic canonicalization is free accuracy — it should key every table, buffer, and dashboard lookup in the project regardless of any other choice. One caveat travels with it: the 19.3× shrinks as board and draw cards pin suits down — late-street symmetry is weaker, and Waugh's perfect-indexing algorithm [15] is the standard way to key the canonical classes.

E[HS] — expected hand strength: equity vs. a random opponent hand, averaged over runouts

Layer 1 — a scalar per hand. Rank every hand by **E[HS]** and cut into percentile buckets. Trivial to compute, fine for a first prototype — and blind in a specific, poker-relevant way: a made hand and a big draw can carry the *same* average equity with *opposite* distributions (Figure 5), and they want opposite strategies (one wants the pot now; the other prices in cards to come).

earth mover's distance — distance between histograms = minimum "work" to shovel one shape into the other

imperfect recall — let the abstraction forget earlier-street details to spend its bucket budget on the present

Layer 2 — the distribution. Represent each hand by its full equity *histogram* over runouts and cluster the histograms with k-means under **earth mover's distance** (Johanson et al. 2013 [16]). Distribution-aware buckets beat scalar buckets on both exploitability and head-to-head play. The same study validated **imperfect recall** — strictly better use of a fixed budget, at the cost of voiding CFR's convergence guarantee (it works empirically; Section 1's perfect-recall condition is the reason the theory breaks).

Layer 3 — the trajectory. Two hands can even share a final histogram but *realize*

it differently — exactly the pre-draw/post-draw distinction. Potential-aware abstraction (Ganzfried & Sandholm 2014 [18], the champion-agent method) represents an early-street hand by its distribution over *next-street buckets*, built bottom-up, so when strength arrives matters, not just how much.

Drawmaha translation: pre-draw buckets should be distributions over post-draw buckets — one recursive level, since there is one draw. This is literally the situation the method was invented for.

Actions get abstracted too. A pot-limit tree still needs a bet-size menu (the project's v1: {pot} only, by design), and any *deployed* strategy must interpret off-menu sizes. The standard is the **pseudo-harmonic mapping** (Ganzfried & Sandholm 2013 [17]): randomize between the neighboring menu sizes with game-theoretically derived probabilities — naive nearest-size rounding is provably exploitable by bet sizing alone. The dashboard needs this the day it accepts a user-typed bet; the principled upgrade is re-solving off-menu spots exactly (Section 9).

action translation — rule mapping a real-world bet onto the solver's nearest menu sizes

Two warnings temper the whole enterprise. First, the **abstraction pathology** (Waugh et al. 2009 [14]): refining an abstraction — more buckets, finer distinctions — can make the real-game strategy MORE exploitable; quality is non-monotonic in size, so bucket sweeps must be validated by measured exploitability, never assumed. Second, the modern question — *doesn't deep learning make all this obsolete?* — gets a precise answer: Deep CFR's paper claims it "obviates the need for abstraction," and what it actually removes is the hand-crafted *clustering*. The *representation* problem survives inside the input featurization: canonical hands, card embeddings, street and pot features are abstraction moved into the network's first layer, and everything this section learned — distribution shape matters, trajectory matters, and for Drawmaha strength is **two-dimensional** (inner equity, outer equity, scoop probability, so a scalar is even more misleading than in hold'em) — is the checklist for those features.

For this project abstraction therefore lives in three places: free canonicalization everywhere; honest bucketing (layers 1–3) for the tabular mini-drawmaha stage; and feature design plus the dashboard's human-readable buckets at full scale.

The display side reuses this machinery: an equity-histogram k-means into 8–20 *named* buckets over the 134,459 classes is the planned range-view grouping, even though the solver itself is bucket-free.

8 Deep CFR: the ledger becomes a regression problem

DEEP CFR

Now the pieces assemble into the project's centerpiece. At full Drawmaha scale no table fits, and **Deep CFR** (Brown, Lerer, Gross & Sandholm 2019 [20]) makes the one substitution left to make: everything about CFR stays — external-sampling traversals, regret matching, average-vs-current — but the per-infoset ledger is replaced by a network trained to *predict* what the ledger would contain, generalizing across the millions of hands that were never sampled. The bet is statistical: similar hands should regret alike, so every sample teaches a neighborhood, and bucketing is learned end-to-end instead of hand-engineered. It was the first non-tabular CFR to work in large poker, beating NFSP on exploitability (37 vs. 47 mb/hand on Flop Hold'em) and playing competitively against a 3.3×10^8-bucket abstraction in full limit hold'em.

Deep CFR — CFR where neural networks predict the regret ledger's contents instead of storing them

Walking Figure 6 clockwise:

1. **Traverse.** Run external-sampling traversals of the *real* game (Section 6); the current strategy at any infoset is regret matching over the advantage network's predictions.
2. **Collect.** Each traversal writes (infoset features, sampled instantaneous regret, iteration t) tuples — **advantages** — into a per-player buffer, and (infoset, current strategy, t) into a strategy buffer.
3. **Regress.** Each iteration trains that player's advantage network *from scratch* on its buffer. From scratch is not paranoia: cumulative regrets are nonstationary, and the paper's ablation shows fine-tuning one network across iterations

advantage (CFR sense) — a sampled instantaneous counterfactual regret; the regression target, NOT actor–critic's advantage

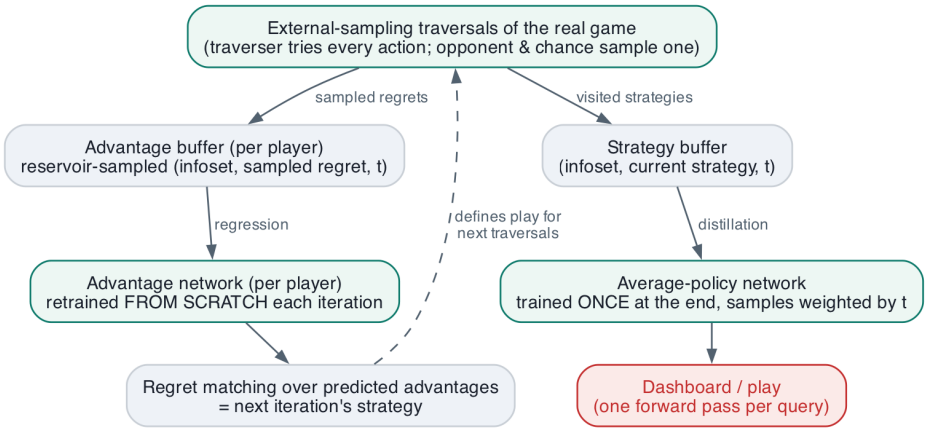


Figure 6: The Deep CFR training loop; teal = learned components, red = what the dashboard queries.

converges substantially worse.

4. **Average.** At the very end, one **average-policy network** is distilled from the strategy buffer with samples weighted by t (Linear-CFR weighting, Section 5). That single network IS the solver output — the thing the folk theorem blesses, the thing the dashboard queries at one forward pass per hand.

average-policy network — a net distilled from all iterations' strategies, weighted by t — the deployable equilibrium approximation

> INTUITION — **reservoir buffers: a fair fixed-size memory of an endless stream**
>
> The buffers cannot hold every sample ever generated, and a FIFO buffer would remember only recent iterations — but the *average* strategy needs all of history represented fairly. Reservoir sampling keeps a fixed-size buffer that is, at every moment, a uniform random sample of everything seen so far: each new item displaces a random resident with exactly the right probability. Deep CFR's guarantees lean on this fairness; swapping in a FIFO buffer is a known way to quietly break convergence.

```
for t in 1..T:                                  # Deep CFR outer loop
  for p in players:
    for k in 1..K:                              # e.g. K = 10,000 traversals
      traverse(deal(), p, adv_net[p], adv_buf[p], strat_buf, t)
    adv_net[p] = train_from_scratch(adv_buf[p],   # regression on
                   target="sampled regret", weight="t")  # ledger contents
final = train(strat_buf, target="strategy", weight="t")  # average-policy net
# play/dashboard: sigma(I) = regret_match(final(I))  -- one forward pass
```

The published recipe, as starting hyperparameters

Deep CFR's appendix is a de-risked starting point, small enough to be encouraging:

Network	7 layers, ~99K parameters: card branch + bet branch
Card encoding	rank + suit + card embeddings, summed within permutation-invariant groups
Bet encoding	per betting slot: occurred-flag + bet size as pot fraction
Buffers	40M samples each (advantage ×2, strategy), reservoir-sampled
Per iteration	10K traversals; retrain 4K SGD steps, batch 10K
Optimizer	Adam, lr 10^{-3}, gradient-norm clip 1
Small-game anchor	(SD-CFR, Leduc): 3×64 nets, 1M buffers, 1.5K traversals/iter

The card-embedding scheme transfers to Drawmaha almost verbatim — permu-

tation invariance over hole cards only grows in value with 5-card hands — but two encoding questions have *no* published answer and are flagged now as genuine design work: how the 32-subset **discard action** should be headed (flat 32-way output vs. per-card keep/discard logits — regret matching over a factorized action space is unexplored territory), and how both players' public **draw-count histories** (plus the face-up draw-1 card) enter the input features. Section 14 carries both.

The forks in the neural-CFR family

SD-CFR / DREAM / ESCHER

Successors map the design space, and three matter here. **SD-CFR** (Steinberger 2019 [21]) deletes the average-policy network: since the average strategy is just an iteration-weighted mixture of per-iteration strategies, and each is recoverable from that iteration's advantage net, storing every checkpoint (~400KB each) reconstructs the *exact* average — one less approximation, measurably better exploitability. Its cost is inference: querying means evaluating many checkpoints, awkward for a live dashboard. **DREAM** (2020 [22]) and **ESCHER** (2022 [23]) exist for simulator-free settings: outcome sampling forces importance weights, whose variance takes an extra learned baseline network (DREAM) or a learned history-value function under a fixed sampling policy (ESCHER) to control. Their lesson for a project that *owns* its simulator is purely diagnostic: importance-sampling variance is what kills neural CFR at scale — choose external sampling on day one and the whole problem never appears.

Best of both, and the plan: train SD-CFR-style (keep checkpoints, evaluate exactly), distill ONE average-policy net at the very end for serving.

Two pitfalls close the section, both documented failure modes. Forgetting the iteration-t weighting on strategy-buffer samples silently converges to the wrong average — a classic bug, since nothing crashes. And the loss curves of advantage networks are *expected* to look bad: cumulative regrets grow and drift across iterations (the nonstationarity ReCFR [24] formalizes), so a plateauing or climbing regression loss is often target drift, not optimizer failure — one more reason Section 11 insists exploitability curves, never training losses, are the progress signal.

9 The other lineage: search at play time, value networks at the horizon

WHY SEARCH IS HARD HERE

Deep CFR *amortizes*: all solving happens in training, and play is a forward pass. The rival lineage — the one behind DeepStack, Libratus, and today's GTO Wizard AI — *searches*: solve the situation you are actually in, at play time, the way chess engines do. In poker that idea breaks twice before it works. First, you never know which state you are in, so search must reason over *ranges* (Section 1's probability cloud) rather than positions. Second, and less obviously:

> INTUITION — **why you cannot just solve the river when you get there**
>
> How an opponent plays a river depends on rivers that never happened — if your new river strategy stops paying off their bluffs, then *arriving* at this river with bluffs was a mistake for them, and a rational opponent's whole earlier game shifts. Solve the subgame "optimally" in isolation and you can make your *overall* strategy more exploitable. The 2014 fix (Burch et al. [25]) is a buyout clause: re-solve the subgame under a constraint that at its entrance, every opponent hand may either play on into your new strategy or take a "buyout" equal to what that hand was worth under your old strategy. If

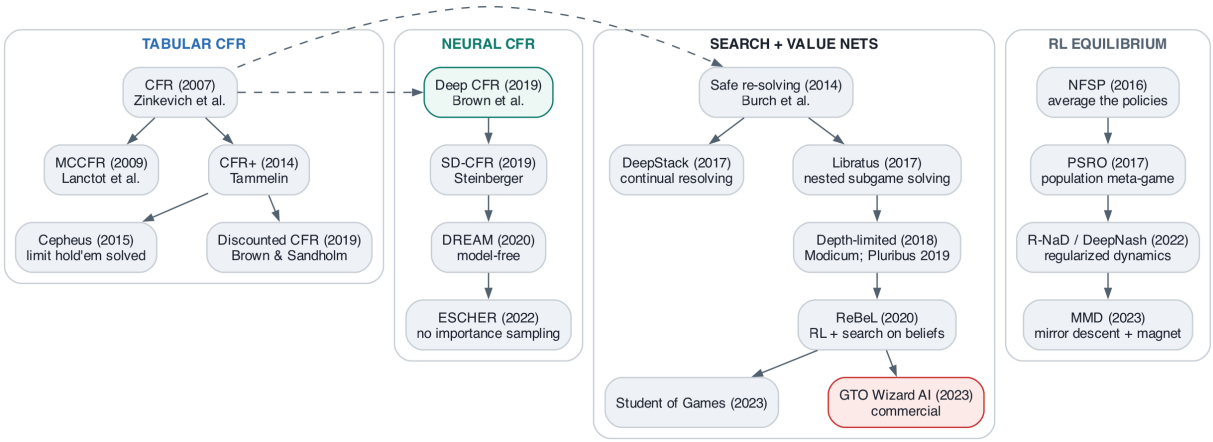


Figure 7: The four method families and their landmark systems; dashed arrows mark CFR's ideas feeding the other lineages.

> no hand prefers deviating, your new strategy concedes nothing it had not already conceded — *safe re-solving*, the gadget every sound search method builds on.

With safety available, the lineage is a sequence of ways to stop searching early (Figure 7 maps it):

- **DeepStack** (2017 [26]): re-solve a small lookahead at *every* decision ("continual resolving"), carrying only your range and the opponent's buyout values forward; where the lookahead ends, a **counterfactual value network** estimates the rest. First expert-level HUNL win; the net's input is both players' full range vectors.
- **Libratus** (2017 [27]): superhuman with *zero* neural networks — an abstraction blueprint (MCCFR), plus *nested* safe re-solving that spawns a finer subgame solve every time the opponent does anything (including bet sizes outside the blueprint's menu, retiring action translation), plus overnight self-repair of the blueprint where opponents probed. Proof that search, not deep learning, was the load-bearing superhuman ingredient.
- **Depth-limited solving** (2018 [28]): the cheap trick — at the depth limit, instead of a value net, let the opponent choose among a handful of precomputed *continuation strategies* (an adversarial "several ways the rest could play"). Modicum beat top prior bots on a 4-core laptop; Pluribus (2019 [29]) rode the same recipe to 6-max on commodity hardware.
- **ReBeL** (2020 [30]): the clean theory — recast poker as a perfect-information game over **public belief states** and run AlphaZero's loop (self-play + search + value/policy nets) on that transformed game, with a Nash convergence proof. **Student of Games** (2023 [31]) generalized the recipe to chess, Go, HUNL, and Scotland Yard with one algorithm.
- **GTO Wizard AI** (commercial, ex-Ruse [32]): the deployed endpoint — solves street-by-street in real time with self-play-trained value networks standing in for future streets; self-reported: arbitrary stacks and bet sizes, ~3s per street, rivers solved exactly, +19.4bb/100 over Slumbot. No paper, so treat all numbers as vendor claims; the *architecture pattern* is the public part.

counterfactual value net — maps (pot, board, both ranges) → per-hand values — a learned evaluation function over ranges

public belief state — public observations + both players' hand distributions: a state everyone CAN see

Amortized vs. search: the actual tradeoff, and the Drawmaha verdict

	Amortized policy (Deep CFR)	Search + values (DeepStack/ReBeL)
Per-query cost	one forward pass, ms	a subgame solve, seconds
Errors at play	baked in at training	partially corrected by search
Off-menu spots	needs action translation	re-solves them natively
Engineering	one training loop	belief tracking + gadgets + nested solves
Range input	never materialized	**2,598,960-dim vectors**

THE 2.6M-DIM WALL

The last row is the Drawmaha-specific wall: every neural system in this lineage feeds *range vectors* into its networks — 1,326 dimensions in hold'em, **2,598,960** in Drawmaha (5.2M for both players). Nothing ports without aggressive hand bucketing or a factorized belief representation first, and that is the single biggest open design problem for any future Drawmaha search layer. Working in search's favor: the public draw count slots perfectly into public-belief-state bookkeeping, pot-limit bounds the bet-size branching, and the post-draw game is comparatively small. The project's verdict, echoing the commercial history: amortized Deep CFR first — search is the accuracy upgrade you buy later, not the foundation you start on. One honest caveat travels with any future resolve layer: updating the opponent's range *through* a hidden discard (they drew two — which two?) is a belief-update computation no paper writes down for poker draws; Section 14 carries it as an open problem.

Ranked future upgrades, cheapest first: (1) depth-limited resolve at the draw boundary using the Deep CFR net ± perturbations as continuation strategies; (2) a post-draw value net, DeepStack-style; (3) full belief-state search.

10 Where actor–critic fits: the RL road to equilibrium, and its potholes

WHY PPO SELF-PLAY FAILS

A reader arriving from applied RL asks the obvious question: why not just PPO in self-play — policy network, value critic, reward = chips, train against a copy of yourself? The answer is a theorem, not a tuning problem. Perolat et al. [33] proved that the idealized dynamics of gradient self-play in two-player zero-sum imperfect-information games are *Poincaré-recurrent*: trajectories **orbit** the equilibrium and return arbitrarily close to where they started, forever. The learning vector field *rotates* around the solution instead of pointing at it (Figure 8, left) — rock–paper–scissors chasing, at scale, in your GPU bill. Cycling in zero-sum self-play is geometry, not a hyperparameter problem: no learning-rate schedule fixes a field with no attraction. The deep reason is Section 1's fact 2: equilibria are *mixed*, and "improve greedily against the current opponent" has no mixed fixed point to settle into.

Contrast the games where naive self-play *does* work — AlphaZero's chess, shogi, Go [34]. Perfect information supplies two pillars: every state has a minimax value independent of beliefs (so a value target exists), and a deterministic optimal policy exists (so greedy improvement is a genuine ladder). Hidden cards demolish both — values depend on ranges, ranges on both strategies, and optimal play must randomize. Everything that works in imperfect information restores convergence by one of three escapes:

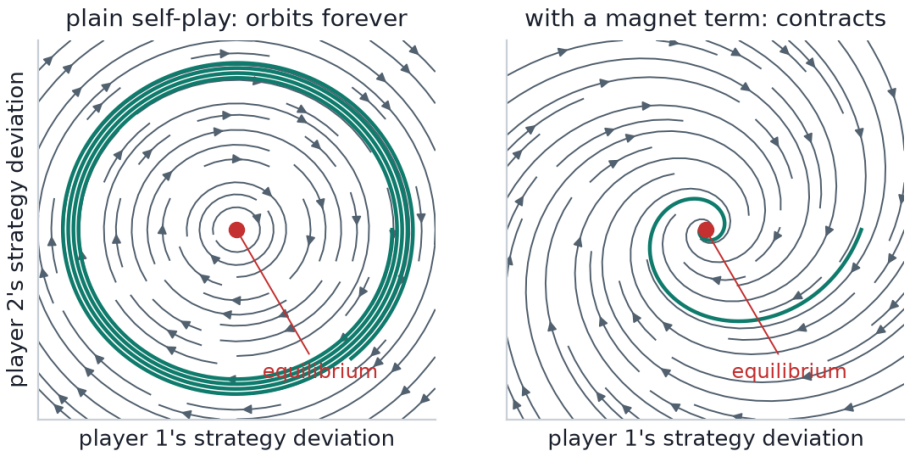


Figure 8: Real trajectories of the two dynamics (simulated for this paper): unregularized self-play orbits; adding a KL pull toward a reference policy makes the same dynamics contract.

Escape	Method	Mechanism, and its cost
Average	NFSP (2016) [35]	DQN best-responds to the opponent's *historical average* policy; a supervised net learns that average and is what you deploy. Cost: slow; beaten by Deep CFR.
Population	PSRO (2017) [36]	meta-game over whole trained policies: solve the small matrix, train a new RL best response to the mixture, repeat. Cost: a full RL run per iteration; coarse at poker's fine mixing.
Regularize	R-NaD / DeepNash (2022) [37]; MMD (2023) [38]	add a KL pull toward a reference "magnet" policy — the regularized game has a *unique* equilibrium the dynamics contract to; move the magnet, repeat. Cost: a regularization schedule; guarantees soften under sampling.

The averaging escape should feel familiar: it is CFR's own trick wearing RL clothes (fictitious play averages policies; CFR averages in regret space). The regularization escape is the modern one — DeepNash reached top-3 human ranking in *Stratego* (10^{535} nodes) model-free, no search, and its "last-iterate" property means the final network itself is the answer, no averaging machinery. MMD is its minimal form — mirror descent plus one extra KL term, the first standard-RL-shaped algorithm empirically competitive with tabular CFR on benchmark games — and the honest note is that its deep-RL evidence so far is small games.

If the project ever wants an RL baseline next to Deep CFR: tabular MMD drops into the same Kuhn/Leduc harness as one extra loss term — the version of "PPO self-play" that has a convergence story.
ALPHAHOLDEM

The honesty file: what PPO actually does in poker

The strongest published PPO-style result is **AlphaHoldem** (2022 [39]): Trinal-Clip PPO (an extra ratio clip when advantages are negative, plus pot-bounded value targets) and a K-best pool of historical selves, trained three days on one PC — beat Slumbot by 111 mbb/hand and pros by 10 mbb/hand, deciding in 3 ms. Tempting. The catch is what head-to-head wins cannot certify: in the Approximate Exploitability study [40], two bots statistically *indistinguishable* head-to-head differed by ~1,300 mbb/hand in measured exploitability. Head-to-head strength and closeness-to-equilibrium are different quantities, and only the second is a solver's claim — for Drawmaha, where no external opponent exists to even run the head-to-head, a PPO bot's quality would be simply unmeasurable. There is also a mechanical reason PPO underperforms here, found only recently [41]: GAE's advantage estimates carry variance from sampling your own future actions, and because equilibrium play is *deliberately* stochastic, that vari-

ance persists even with a perfect critic — expected-SARSA-style traces (VRPO) are the drop-in fix. File both away for the day the project trains its planned PPO baseline.

So where does actor–critic *legitimately* live in this project? Three places, all supporting roles: as the **exploiter** — train a PPO/DQN best response against the *frozen* final network, where the opponent holds still, cycling vanishes, and the exploiter's winnings are an exploitability lower bound (Section 11); as **baselines** inside variance-reduced MCCFR, which are control-variate critics by another name (Section 6); and as the parked **comparison chapter** — CFR solution vs. PPO self-play agent, with the exploiter quantifying the gap the theory predicts.

11 Evaluation: the only honest progress meters

LOSS CURVES LIE

One sentence organizes this whole section: a solver's training loss tells you nothing — self-play generates no ground-truth labels, so the only principled question is "how much would a perfect adversary win against my strategy?", asked with different instruments at different scales. Deep CFR's advantage losses are *expected* to drift (Section 8); OpenSpiel's own Deep CFR has open bug reports caught only by exploitability curves. Every stage of the project needs its instrument chosen in advance.

Units first. The field reports exploitability and win rates in **mbb/hand** — and its calibration anchors are worth knowing *with* their caveat: in hold'em, folding every hand loses 750 mbb/hand, professionals treat ~50 as a big edge, and Cepheus's <1 defines "essentially solved." Those numbers are hold'em-and-blind-structure specific; Drawmaha needs its own always-fold anchor computed, not borrowed.

mbb/hand — milli-big-blinds per hand; 1000 mbb = 1 big blind. bb/100 = mbb/hand ÷ 10

Drawmaha's unit, defined once and used everywhere: total chips won across BOTH halves of the pot, per hand, in big blinds. Split-pot diagnostics ride along: per-half EV and scoop rate.

The instrument ladder

Instrument	What it certifies	Cost	Scale limit
Exact best response	exploitability, exactly	one tree walk	Kuhn → mini-drawmaha
Local best response (LBR)	a *lower bound*	hands vs. the bot	full game
Learned exploiter (RL-BR)	a tighter lower bound	one RL training run	full game
Head-to-head + variance reduction	*relative* strength only	simulated matches	any

Exact best response is a single expectimax walk: the responder maximizes at its nodes, follows your known strategy at yours, expects over chance — and the crucial constraint is that it maximizes per *infoset*, never per deal (an adversary who sees your cards is a different, vacuous number; this paper's own Figure 3 pipeline contained exactly that bug for an hour). Vectorizing over all private hands along the public tree (Section 5) pushed exactness to full limit hold'em — 76 CPU-days — and produced the field's formative shock [42]: bots near-optimal *within their abstractions* were exploitable for hundreds of mbb/hand in the real game. Abstract-game exploitability is not real-game exploitability — and the abstraction pathology (Section 7) makes the gap non-monotonic on top. Project translation: size mini-drawmaha so this exact walk stays affordable — *shrink the deck, not just the stacks*, because the private-hand vector (2.6M vs. 1,326) is what makes the walk expensive.

Local best response (Lisý & Bowling 2017 [43]) is the full-scale probe: play hands against the frozen strategy while tracking the opponent's exact range by Bayes' rule (possible because the strategy is queryable for every hand), and at each de-

cision greedily pick the action maximizing EV under a crude "everyone checks/calls to showdown" rollout. Because LBR is itself a legal strategy, its winnings *certify a lower bound* on exploitability. Its shock result: top HUNL bots within 24 mbb/hand of each other head-to-head were all >3,180 mbb/hand exploitable — worse than always-folding. Two honesty rules attach: a lower bound can be vacuous — LBR has famously LOST to bots that were provably exploitable, so "LBR found nothing" never means "unexploitable"; and for Drawmaha its machinery needs real adaptation (range updates through the draw, a stand-pat analogue of the call-down heuristic) — which is why the modern alternative matters: **train the exploiter** (ISMCTS-BR [40], or a plain DQN/PPO agent) against the frozen net. The opponent holds still, so this is a single-agent RL problem — far easier than solving the game — and it is the planned worst-case probe.

Head-to-head is unavoidable at full scale (checkpoint vs. checkpoint, final bot vs. scaled-up mini-game recipe vs. rule-based baselines) and drowns in variance without help. Two standard tools: **duplicate dealing** — ~30 lines of code, assumption-free, build it into the match runner on day one — and **AIVAT** [44], the stronger correction:

duplicate dealing — play each pre-dealt deck twice with seats swapped; luck cancels by symmetry

> INTUITION — **AIVAT: subtract the luck, keep the skill**
>
> Every chance event (a lucky river) and every known-strategy random action injects variance a value function can anticipate. AIVAT adds, at each such event, a correction term: (expected value just before it) minus (value just after), computed from any heuristic value function. Each term is zero on average — so the estimate stays *unbiased no matter how bad the value function is* — but it cancels the luck the event injected; quality only determines how much variance disappears. It is the control-variate/baseline trick a third time (after VR-MCCFR and GAE), and it bought DeepStack's human study a ~10× sample-size saving. A Monte Carlo pot-equity rollout suffices as the value function.

The pattern to copy, and the project's plan

Deep CFR's own evaluation [20] is the template the field accepts: **tier 1**, a mid-size game where exact exploitability is computable (Flop Hold'em), carrying the scientific weight — convergence curves against baselines; **tier 2**, the full game, where only head-to-head is possible. DeepStack's study adds the structure for claims about strength: variance-reduced match results *plus* an adversarial probe, because a big head-to-head win is provably compatible with huge exploitability. Mapped onto the validation ladder (Figure 10, Section 13): Kuhn/Leduc — exact exploitability, checked against OpenSpiel's reference; mini-drawmaha — exact exploitability as the arena where MCCFR vs. Deep CFR vs. ablations get compared, *plus* running the full Deep CFR pipeline there to measure how far the nets land from the tabular truth; full game — duplicate-dealt checkpoint ladder with AIVAT, plus the learned exploiter as the worst-case meter, reported as a lower bound with the vacuousness caveat stated.

12 Precedents: how each Drawmaha blow-up has been beaten before

PLO PRACTICE

No one has solved Drawmaha, but each of its three blow-ups (Section 2) has been beaten separately, and the precedents set this project's expectations.

The hand-space blow-up: pot-limit Omaha practice. PLO's 270,725 starting hands are the nearest commercial-scale precedent, attacked three ways. **MonkerSolver** (industry standard since ~2017): MCCFR over a bucketed tree — no preflop bucketing, postflop buckets from strength tiers × board texture × blocker classes, granularity a user dial against RAM and days of runtime. Battle-tested, and its artifact is instructive: hands inside one bucket cannot differentiate, so bluffs concentrate into few combos — the bucketing fingerprint. **GTO Wizard AI** solves PLO street-by-street with self-play-trained EV networks replacing future streets, rivers solved exactly (self-reported: Nash distance <0.1% pot on rivers, ~0.14% average EV loss on turns, seconds per spot). **Vision GTO** serves precomputed abstracted solves as a database — the delivery model this project's live-inference dashboard deliberately rejects, for the same reason the market moved on: custom spots explode any precomputed menu.

The river-exact + NN-elsewhere hybrid is copyable at hobby scale: solve the post-draw street exactly; let the network cover the rest.

The draw-round blow-up: the Ostroumov solver. The only detailed public account of an equilibrium solver for a real draw game [46]: limit 2-7 triple draw, built 2013–15 for high-stakes pros and used to prepare the 2015 Kuznetsov–Ivey match. The full game would not fit, so: handcraft preflop and first-draw play, solve every post-first-draw subtree independently (the way hold'em solvers treat flops), iterate the handcrafted trunk against the computed EVs by hand. One full run: ~1,000 CPU cores for 3 days. Its two transferable lessons: subgame decomposition around the draw works, and — the upgrade its author called the biggest — tracking even ONE discarded card changed strategies materially: discard blockers (you threw the card their draw needs) are real EV, so discard identity must survive in the infoset even though only the count is public. Snowing — standing pat on nothing as a bluff — only exists if the representation can express it. The cautionary twin is **Poker-CNN** (2015 [48]), which learned 2-7 triple draw draws and bets by self-play CNN with the opponent's draw count as an input feature — it works as a *bot*, and its own author flags the gap: no internal consistency, no equilibrium claim. Exactly the Deep-CFR-vs-plain-self-play distinction of Section 10, played out in a draw game. And the oldest anchor still pays rent: Zadeh's 1977 analytical treatment of five-card draw [49] derives pre- and post-draw bluffing frequencies from pot odds — closed-form sanity checks a toy-drawmaha CFR run should reproduce.

The split-pot blow-up: the Ho recipe. A Harvard master's thesis [47] computed an ε-equilibrium for the jam/fold endgame of no-limit Omaha Hi-Lo — four cards, split pot — with CFR+ on cloud GPUs, and its methodology chapter is effectively this project's split-pot engineering spec: enumerate all starting hands via a combinatorial index; **bit-pack the high rank and low rank into one integer so a single comparison settles both halves**; pre-tabulate win/split/scoop outcomes over suit-isomorphic board classes (152,620 of them); and put every drop of split-pot logic in the terminal payoff function so the CFR core runs unmodified — the split pot never touches the algorithm, only the leaves. MonkerSolver's Omaha-8 support confirms the same at commercial scale. One Ho-specific caveat: his all-pairs matchup precomputation squares badly from 4-card to 5-card hands — lazy or sampled evaluation replaces it at Drawmaha size.

NEAREST NEIGHBORS

Two adjacent literatures round out the map. Gin rummy (the AAAI/EAAI challenge [50]) is the academic home of draw/discard games with *public* discards — one entry learns a value function for the combinatorial discard choice while solving the small endgame exactly with MCCFR, a plausible architecture split for mini-drawmaha — though its information structure is easier (discard identities are public; Drawmaha hides them). And the absence finding bears repeating

with its consequence: with no Drawmaha bot anywhere, *every* baseline in the evaluation plan is homegrown, so the checkpoint ladder and rule-based agents of Section 11 are not conveniences — they are the entire comparative universe until someone else builds one.

13 Engineering: the toolkit, the bug catalog, and the ladder

THE TOOLKIT

The entry barrier to this field has fallen steadily — a solo engineer can now reproduce Deep-CFR-class results — but the tools have sharply different roles, and the sweep behind this survey found the community's hard-won bug list scattered across tutorials, ablations, and GitHub issues. Both are collected here.

Tool	Role in this project	Trust level
OpenSpiel [51]	the *referee*: reference CFR + exact exploitability on Kuhn/Leduc; verify your numbers against it	tabular code: high; its Deep CFR: has open bug reports
PokerRL [52]	the *architecture reference*: Deep CFR/SD-CFR + a graded BR→LBR→RL-BR evaluation ladder, by SD-CFR's author	read and port from; frozen on PyTorch 0.4 — do not depend on
PokerKit [54]	the *rules oracle*: variant poker (draws, splits) as configuration, 99% test coverage; unit-test the custom engine against it	good; pure-Python speed, so not the inner loop
RLCard [53]	quick sandbox, clean encoding patterns	fine for prototypes; weak equilibrium tooling

Hand evaluation sits on the critical path: every MCCFR terminal node needs *two* evaluations (inner hand, outer hand), millions of times per run. The classic lineage solves it: all 5-card hands collapse to exactly 7,462 distinct strength classes (Cactus Kev), reachable by perfect-hash lookup in a handful of memory reads [55]; open C/C++ evaluators (OMPEval, PokerHandEvaluator with its Omaha mode) are 10–100× faster than Python ones. Since Drawmaha hands are exactly five cards, the 5-card perfect hash fits natively; Ho's bit-packing (Section 12) makes the split-pot comparison one integer operation.

No library implements Drawmaha itself — the split-pot draw engine is genuinely custom work; the tools above check it rather than replace it.

The bug catalog

Five bugs account for most broken CFR implementations in the community record; all are *silent* — nothing crashes, the answer is just wrong:

The bug	The rule that prevents it
Reach-probability swap	regrets weight by *opponent+chance* reach; average-strategy accumulation weights by *own* reach — never the reverse (Section 4)
Querying the current strategy	every guarantee is about the *average*; vanilla CFR's current strategy cycles forever (Fig. 3)
Missing iteration weights	linear averaging means weighting by t — in tabular sums *and* in Deep CFR's buffer samples
Missing sampling correction	outcome sampling must divide by the sample probability; external sampling needs no correction — one reason to prefer it
FIFO instead of reservoir	Deep CFR's buffers must be uniform over *all* iterations; recency-biased buffers break the average

The antidote to all five is the same discipline: regression-test against known values at every stage — Kuhn's closed-form equilibrium and game value ($-1/18$), Leduc's sequence-form-LP value ($-0.085606\ldots$ for player 1), and OpenSpiel's exploitability curves reproduced iteration-for-iteration. This paper's own Figure 3

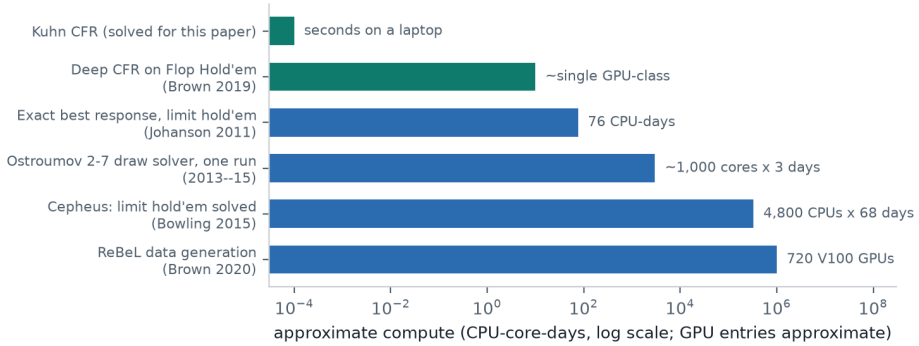


Figure 9: Approximate compute of landmark systems, from reported hardware (log scale; order-of-magnitude estimates).

pipeline was validated exactly this way (and Section 11 confessed the bug the check caught).

Compute reality, and the ladder that structures the build

Figure 9 places the landmark systems on one axis, and it doubles as the argument for the project's scope: the tabular triumphs (Cepheus) and the search flagships (ReBeL) sit orders of magnitude beyond a solo budget, while Deep CFR's published recipe is single-GPU-class — *the* reason it is the chosen endgame.

Modal GPUs cover the Deep CFR phase if the laptop doesn't; nothing in the plan needs a cluster.

The build itself follows the field's standard validation practice — prove each layer on a game with a known answer before adding the next — instantiated for this project as Figure 10's ladder. Its logic: Deep CFR has three separately-breakable layers (the CFR core, the Drawmaha rules engine, the networks); climbing one rung at a time means every failure has one suspect. The mini-drawmaha rung earns special emphasis: it is where all three Drawmaha-specific mechanisms (split pot, draw round, face-up draw-1) get proven while exact evaluation is still possible — and it doubles later as tier 1 of the full evaluation (Section 11).

14 The decision checklist: every fork this project will actually face

THE CHECKLIST

This section is the survey folded into a working document: every design decision the project will meet, bucketed, each with the literature's default and the section that justifies it. Items marked ◦ have *no* published answer — they are the genuinely novel work.

A. Game specification (settle before writing the engine)

- Rules pinned as in Section 2: HU, pot-limit, {fold, check/call, pot} v1 menu as config, 100bb default, one draw post-flop, face-up draw-1 rule, cards-speak split with both-halves tie table written out (including odd-chip handling). Target rake-free play (rake shifts the equilibrium; state the choice).
- Infoset key contents: own 5 cards (canonicalized) + board + full betting history + **both players' draw-count histories** + own discard identities + the face-up draw-1 card when present (Sections 2, 12).
- Scope statements, written once in the README: all guarantees are heads-up (3+ players voids interchangeability and safety, Section 1; the practical stance elsewhere — Pluribus — is train HU, accept heuristic multiway play); equilibrium only, no opponent modeling/exploitation (a different literature: restricted Nash response, safe exploitation).

B. Representation and encoding

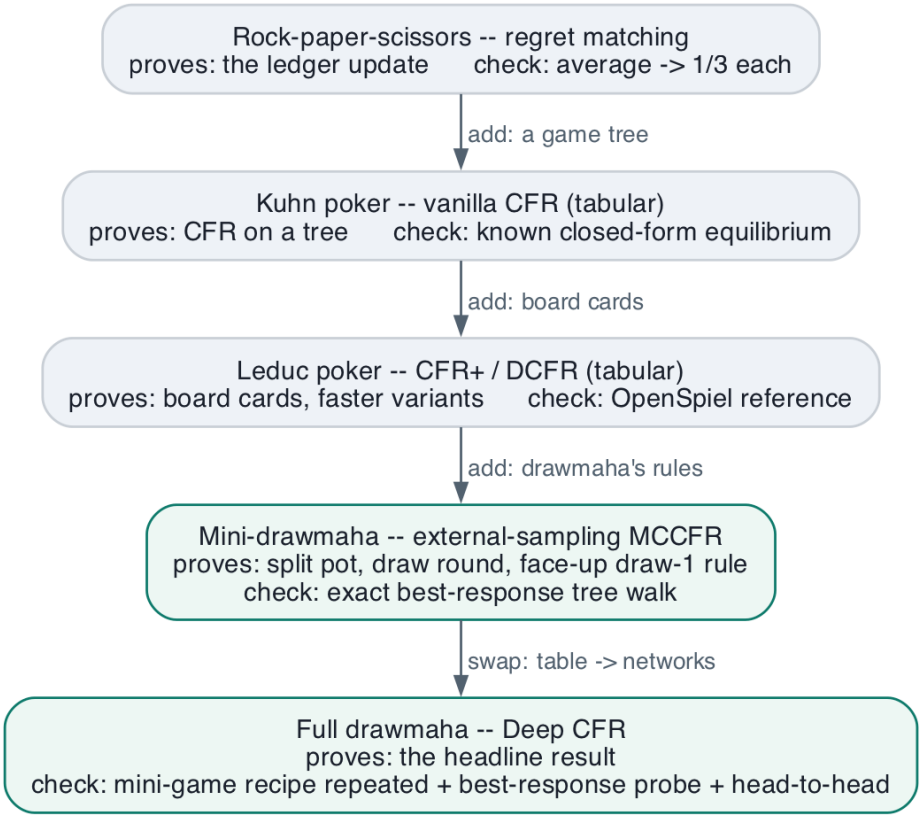


Figure 10: The project's validation ladder: one new layer per rung, a known answer at every rung.

- Suit-isomorphic canonicalization (134,459 classes) keys everything; Waugh-style perfect indexing; expect weaker symmetry postflop (Section 7).
- Card inputs: Deep CFR's summed rank/suit/card embeddings, permutation-invariant over hole cards (Section 8). Betting history: per-slot occurred-flag + pot-fraction features to start.
  - Discard action head: flat 32-way softmax vs. per-card factorized logits — unexplored; start flat (regret matching over 32 actions is standard), benchmark factorized later.
  - Draw-signal features: both draw counts, the face-up card, and kept-card identities — feature design with no citation; validate on mini-drawmaha ablations.
- Dual evaluator: 5-card perfect hash for the inner hand + Omaha-mode evaluation for the outer; bit-packed ranks; all split logic in terminal payoffs only (Sections 12, 13).

C. Training pipeline

- The ladder (Fig. 10): vanilla CFR → CFR+/DCFR$(1.5, 0, 2)$ on toys; external-sampling MCCFR with linear weighting on mini-drawmaha; Deep CFR at full scale with the published hyperparameters as anchors (Section 8).
- SD-CFR-style checkpointing during training, one distilled average-policy net at the end for serving.
- Escalation valves, in order: VR-MCCFR baselines if sampling noise binds; robust sampling ($k <$ all) if the 32-subset draw node makes traversals slow; ESCHER-style value function if advantage-target variance won't settle; depth-limited resolve at the draw boundary as the accuracy upgrade (Section 9).
  - Mini-drawmaha's exact design (deck size, hand size, streets) — sized so the exact best-response walk is affordable: shrink the *deck*, not just stacks.

D. Evaluation protocol (pre-registered, Section 11)

- Units: total-chips-both-halves per hand in bb; compute Drawmaha's own always-fold anchor; per-half EV and scoop-rate diagnostics.
- Instruments per rung: OpenSpiel-verified exact exploitability (toys); exact walk + Deep-CFR-vs-tabular gap (mini); duplicate-dealt AIVAT head-to-head ladder + trained exploiter, reported as a lower bound (full).
- Match runner ships with duplicate dealing from day one; every claimed difference carries a bootstrap CI.

E. Deployment and dashboard

- Serving: one forward pass per query on the distilled net; per-node range views computed by batch-evaluating all (canonical) hands with reach probabilities from policy products along the line; memoize visited nodes.
- Post-processing before display/play: **purification** and thresholding (Ganzfried & Sandholm 2012) — the standard step between "trained net" and "served strategy."
- User-typed bet sizes: pseudo-harmonic action translation now; nested re-solving later (Section 7).
- Display bucketing: equity-histogram k-means into 8–20 named buckets over canonical classes, filterable by inner/outer hand (the project's chosen UI model).

purification — round near-zero action probabilities away before serving; cuts approximation-noise exploitability

F. Open problems worth a writeup of their own

- Belief updates through a hidden discard (marginalizing over which cards were kept given only the count) — required by any future search layer, written down nowhere.
- A 2.6M-hand range representation for value nets (bucketed or factorized) — the wall between this project and the DeepStack/ReBeL lineage.
- LBR adapted to draw games (range tracking through draws, stand-pat heuristics) — the trained exploiter sidesteps it, but the adaptation would be a contribution.

Nothing in this list is speculative: every default traces to a cited result, and every ◦ is a place where this project, by existing, produces the first data point.

Reading order for the build, matched to rungs: Neller & Lanctot's tutorial (rung 0–1) → Lanctot 2009 (rung 3's MCCFR) → Brown 2019 Deep CFR + Steinberger's SD-CFR/PokerRL code (rung 4) → Lisý & Bowling + Burch AIVAT (evaluation) → the rest as needed.

15 Key takeaways

1. **The target is a number.** Solving means driving *exploitability* — what a perfect adversary wins against you — toward zero. It is computable exactly on small games, lower-boundable on big ones, and it, not any loss curve, is the project's progress bar.
2. **One rule, one theorem.** Regret matching (play proportional to positive regret) plus the folk theorem (vanishing average regret ⇒ the *average* strategy pair approaches Nash) generate the entire CFR family. The average-vs-current distinction is the field's most common bug and the reason an average-policy network is the deliverable.
3. **The project's spine is standard and single-GPU-sized.** External-sampling MCCFR with linear weighting, scaled to Deep CFR with the published hyperparameters as anchors, SD-CFR checkpointing during training, one distilled net for serving. Every landmark that needed more compute (Cepheus, ReBeL) is the argument for this choice, not against it.
4. **Search is the upgrade, not the start.** The DeepStack/ReBeL/GTO-Wizard lineage buys accuracy and custom-spot flexibility at seconds per query — and its range-vector inputs hit a 2.6M-dimension wall in Drawmaha. Depth-limited resolving at the draw boundary is the one cheap borrowing.

5. **Plain self-play RL is a trap here, and provably so.** The dynamics orbit; nothing converges; head-to-head wins cannot certify equilibrium quality. Actor–critic earns its place in three supporting roles: the exploiter, the variance baselines, and the comparison chapter.
6. **Drawmaha's novelties are locatable.** Split pot: terminal payoffs only — cheap. Hand space: canonicalize (134,459 classes) and let networks generalize. Draw round: keep discard identity in the infoset, feed the public counts to the net, and expect the encoding decisions (◦ items, Section 14) to be the project's first genuinely novel results.
7. **Validate like the field does.** One new layer per rung, a known answer at every rung, regression constants from Kuhn/Leduc/OpenSpiel, and a pre-registered evaluation protocol — the practices that separate "I trained a bot" from "I can bound how far it is from equilibrium."

Provenance. Synthesized August 28, 2026 from an eight-lane automated literature sweep (Semantic Scholar, arXiv, and the open web; ~70 sources verified by fetch) plus a completeness audit; lane digests with per-paper notes live in the project repository under `docs/research/`. Claims the sweep could not verify in-session are flagged in those digests; the cited originals remain ground truth. The Kuhn, RPS, and dynamics figures were computed live by code in `paper/make_figures.py`, validated against Kuhn's closed-form game value.

References

[1] S. Hart, A. Mas-Colell. A simple adaptive procedure leading to correlated equilibrium. *Econometrica* 68(5), 2000.

[2] M. Zinkevich, M. Johanson, M. Bowling, C. Piccione. Regret minimization in games with incomplete information. *NIPS*, 2007.

[3] T. Neller, M. Lanctot. An introduction to counterfactual regret minimization. AAAI Model AI Assignments, 2013. `modelai.gettysburg.edu/2013/cfr`.

[4] M. Lanctot, K. Waugh, M. Zinkevich, M. Bowling. Monte Carlo sampling for regret minimization in extensive games. *NIPS*, 2009.

[5] O. Tammelin. Solving large imperfect information games using CFR+. arXiv:1407.5042, 2014.

[6] O. Tammelin, N. Burch, M. Johanson, M. Bowling. Solving heads-up limit Texas hold'em. *IJCAI*, 2015.

[7] M. Bowling, N. Burch, M. Johanson, O. Tammelin. Heads-up limit hold'em poker is solved. *Science* 347(6218), 2015.

[8] N. Brown, T. Sandholm. Solving imperfect-information games via discounted regret minimization. *AAAI*, 2019.

[9] N. Brown, T. Sandholm. Regret-based pruning in extensive-form games. *NIPS*, 2015.

[10] M. Johanson, N. Bard, M. Lanctot, R. Gibson, M. Bowling. Efficient Nash equilibrium approximation through Monte Carlo counterfactual regret minimization / public chance sampling. *AAMAS*, 2012.

[11] D. Koller, N. Megiddo, B. von Stengel. Efficient computation of equilibria for extensive two-person games. *Games and Economic Behavior* 14, 1996.

[12] M. Schmid, N. Burch, M. Lanctot, M. Moravcik, R. Kadlec, M. Bowling. Variance reduction in Monte Carlo CFR using baselines. *AAAI*, 2019.

[13] A. Gilpin, T. Sandholm. Lossless abstraction of imperfect information games. *Journal of the ACM* 54(5), 2007.

[14] K. Waugh, D. Schnizlein, M. Bowling, D. Szafron. Abstraction pathologies in extensive games. *AAMAS*, 2009.

[15] K. Waugh. A fast and optimal hand isomorphism algorithm. AAAI Workshop on Computer Poker, 2013.

[16] M. Johanson, N. Burch, R. Valenzano, M. Bowling. Evaluating state-space abstractions in extensive-form games. *AAMAS*, 2013.

[17] S. Ganzfried, T. Sandholm. Action translation in extensive-form games with large action spaces: axioms, paradoxes, and the pseudo-harmonic mapping. *IJCAI*, 2013.

[18] S. Ganzfried, T. Sandholm. Potential-aware imperfect-recall abstraction with earth mover's distance. *AAAI*, 2014.

[19] S. Ganzfried, T. Sandholm, K. Waugh. Strategy purification and thresholding: effective non-equilibrium approaches for playing large games. *AAMAS*, 2012.

[20] N. Brown, A. Lerer, S. Gross, T. Sandholm. Deep counterfactual regret minimization. *ICML*, 2019.

[21] E. Steinberger. Single deep counterfactual regret minimization. arXiv:1901.07621, 2019.

[22] E. Steinberger, A. Lerer, N. Brown. DREAM: deep regret minimization with advantage baselines and model-free learning. arXiv:2006.10410, 2020.

[23] S. McAleer, G. Farina, M. Lanctot, T. Sandholm. ESCHER: eschewing importance sampling in games by computing a history value function to estimate regret. *ICLR*, 2023.

[24] W. Liu, B. Li, J. Togelius. Model-free neural counterfactual regret minimization with bootstrap learning. *IEEE Transactions on Games*, 2022.

[25] N. Burch, M. Johanson, M. Bowling. Solving imperfect-information games using decomposition. *AAAI*, 2014.

[26] M. Moravcik, M. Schmid, N. Burch, V. Lisy, D. Morrill, N. Bard, T. Davis, K. Waugh, M. Johanson, M. Bowling. DeepStack: expert-level artificial intelligence in heads-up no-limit poker. *Science* 356(6337), 2017.

[27] N. Brown, T. Sandholm. Superhuman AI for heads-up no-limit poker: Libratus beats top professionals. *Science* 359(6374), 2018. (Nested subgame solving: *NIPS* 2017.)

[28] N. Brown, T. Sandholm, B. Amos. Depth-limited solving for imperfect-information games. *NeurIPS*, 2018.

[29] N. Brown, T. Sandholm. Superhuman AI for multiplayer poker. *Science* 365(6456), 2019.

[30] N. Brown, A. Bakhtin, A. Lerer, Q. Gong. Combining deep reinforcement learning and search for imperfect-information games. *NeurIPS*, 2020.

[31] M. Schmid et al. Student of Games: a unified learning algorithm for both perfect and imperfect information games. *Science Advances* 9, 2023.

[32] GTO Wizard. GTO Wizard AI explained; How solvers work; How GTO Wizard solved PLO. Company engineering blog, 2023–2026 (vendor claims, no peer review). `blog.gtowizard.com`.

[33] J. Perolat, R. Munos, J.-B. Lespiau, et al. From Poincaré recurrence to convergence in imperfect information games: finding equilibrium via regularization. *ICML*, 2021.

[34] D. Silver et al. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. arXiv:1712.01815, 2017; *Science* 362, 2018.

[35] J. Heinrich, D. Silver. Deep reinforcement learning from self-play in imperfect-information games. arXiv:1603.01121, 2016.

[36] M. Lanctot, V. Zambaldi, A. Gruslys, et al. A unified game-theoretic approach to multiagent reinforcement learning. *NeurIPS*, 2017.

[37] J. Perolat, B. De Vylder, D. Hennes, et al. Mastering the game of Stratego with model-free multiagent reinforcement learning. *Science* 378, 2022.

[38] S. Sokota, R. D'Orazio, J.Z. Kolter, N. Loizou, M. Lanctot, I. Mitliagkas, N. Brown, C. Kroer. A unified approach to reinforcement learning, quantal response equilibria, and two-player zero-sum games. *ICLR*, 2023.

[39] E. Zhao, R. Yan, J. Li, K. Li, J. Xing. AlphaHoldem: high-performance artificial intelligence for heads-up no-limit poker via end-to-end reinforcement learning. *AAAI*, 2022.

[40] F. Timbers, N. Bard, E. Lockhart, M. Lanctot, M. Schmid, N. Burch, J. Schrittwieser, T. Hubert, M. Bowling. Approximate exploitability: learning a best response in large games. *IJCAI*, 2022.

[41] Z. Fan, G. Farina. GAE falls short in imperfect-information self-play reinforcement learning. arXiv preprint, 2026 (unreviewed).

[42] M. Johanson, K. Waugh, M. Bowling, M. Zinkevich. Accelerating best response calculation in large extensive games. *IJCAI*, 2011.

[43] V. Lisy, M. Bowling. Equilibrium approximation quality of current no-limit poker bots (local best response). AAAI Workshop on Computer Poker, 2017.

[44] N. Burch, M. Schmid, M. Moravcik, M. Bowling. AIVAT: a new variance reduction technique for agent evaluation in imperfect information games. *AAAI*, 2018.

[45] N. Bard, J. Hawkin, J. Rubin, M. Zinkevich. The annual computer poker competition. *AI Magazine* 34(2), 2013.

[46] O. Ostroumov. How I made my first million: story of Draw Solver (2-7 triple draw / badugi). First-person engineering account, GipsyTeam/Medium, 2025 (practitioner source).

[47] K. Ho. A no-limit Omaha hi-lo poker jam/fold endgame equilibrium. Master's thesis, Harvard Extension School, 2015.

[48] N. Yakovenko, L. Cao, C. Raffel, J. Fan. Poker-CNN: a pattern learning strategy for making draws and bets in poker games. *AAAI*, 2016.

[49] N. Zadeh. Computation of optimal poker strategies. *Operations Research* 25(4), 1977.

[50] B. Goldstein, J.-P. Astudillo Guerra, E. Haigh, B. Cruz Ulloa, J. Blum. Extracting learned discard and knocking strategies from a gin rummy bot. *AAAI/EAAI*, 2021.

[51] M. Lanctot, E. Lockhart, et al. OpenSpiel: a framework for reinforcement learning in games. arXiv:1908.09453, 2019.

[52] E. Steinberger. PokerRL (with Deep-CFR/SD-CFR reference implementations). `github.com/EricSteinberger/PokerRL`, 2019.

[53] D. Zha, K.-H. Lai, Y. Cao, S. Huang, R. Wei, J. Guo, X. Hu. RLCard: a toolkit for reinforcement learning in card games. AAAI-20 Workshop, 2019.

[54] J. Kim. PokerKit: a comprehensive Python library for fine-grained multi-variant poker game simulations. *IEEE Transactions on Games*, 2023.

[55] K. Suffecool (Cactus Kev); P. Senzee; and descendants (OMPEval, PokerHandEvaluator). Fast 5-card hand evaluation via 7,462 equivalence classes and perfect hashing. Community engineering lineage, 2006–. `suffe.cool/poker/evaluator.html`.